Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Структури даних та алгоритми (СДА)

Лабораторна робота №3

«Стеки та черги»

Виконав:
студент групи ІО-64
Батиченко Артем Олександрович
Номер у списку групи: 2
Перевірив:
Русінов Володимир Володимирович

Київ – 2026

Мета:

Засвоєння теоретичного матеріалу та набуття практичних навичок реалізації й застосування абстрактних структур даних **стек** (LIFO — «останнім прийшов, першим вийшов») та **черга** (FIFO — «першим прийшов, першим вийшов»), а також порівняння двох способів їхньої реалізації — на основі масиву фіксованого розміру та на основі однозв'язного списку.

Завдання:

1. Кожен варіант завдання визначає прикладну задачу (наприклад, перевірку збалансованості дужок, обчислення постфіксного виразу тощо) та структуру(и) даних, потрібну для її розв'язання (стек, чергу, або обидві).

2. Для розв'язання задачі написати дві програми, що реалізують ту саму структуру даних двома способами:
   i. на основі одновимірного масиву фіксованого розміру (з окремими змінними/індексами для вершини стека або голови й хвоста черги);
   ii. на основі однозв'язного списку (динамічне виділення пам'яті: вузол зі значенням і вказівником на наступний вузол).

   Прикладна логіка (сама задача варіанта) в обох програмах має бути однаковою — відрізнятися має лише внутрішня реалізація стека/черги.

3. Обидві реалізації мають надавати єдиний узгоджений набір функцій, наприклад для стека: push, pop, peek, is_empty, is_full; для черги: enqueue, dequeue, peek_front, is_empty, is_full. Прикладна логіка (main і функції задачі) має викликати ці функції однаково незалежно від того, яка реалізація підключена — тобто дві програми відрізняються лише файлом/блоком, де реалізовані самі функції структури даних.

4. Розмір масиву для масивної реалізації взяти самостійно, достатнім для тестових прикладів (наприклад, 50–100 елементів), і оформити як іменовану константу.

5. Масивна реалізація черги обов'язково має бути кільцевою (циклічною): після заповнення останньої комірки масиву черга повинна продовжувати використовувати комірки, звільнені операціями dequeue, починаючи знову з початку масиву, а не вважати масив вичерпаним.

Варіант 2:

Стеком перевірити правильність вкладення дужок одного виду () у рядку та додатково визначити **максимальну глибину вкладення** (найбільшу кількість одночасно відкритих дужок).

Текст програми 1:

```
#include <stdio.h>
#include <windows.h>

typedef struct
{
   char main[50];
   int lp;
} Stack;

void init(Stack *s) { s->lp = 0; }

int is_full(const Stack *s) { return s->lp == 50; }

int is_empty(const Stack *s) { return s->lp == 0; }

int pop(Stack *s)
{
   if (is_empty(s))
   {
      return 0;
   }
   s->lp--;
   s->main[s->lp] = '\0';
   return 1;
}

int push(Stack *s, char x)
{
   if (is_full(s))
   {
      return 0;
   }
   s->main[s->lp] = x;
   s->lp++;
   s->main[s->lp] = '\0';
   return 1;
}
```

```
int peek(const Stack *s, char *x)
{
   if (is_empty(s))
   {
      return 0;
   }
   *x = s->main[s->lp - 1];
   return 1;
}

int main()
{
   SetConsoleOutputCP(65001);

   Stack stack;

   // char s[]={'(', '(', '(', ')', '(', ')', ')',')','\0'};
   char s[] = "a*(b-c:((d+e)*(f+g)))-h*(k-l)+m";

   int max = 0, chek = 0;

   init(&stack);

   printf("Рядок для перевірки: ");
   puts(s);
   printf("\n");

   for (int i = 0; s[i] != '\0'; i++)
   {
      if (s[i] == '(')
      {
         chek = push(&stack, s[i]);
         if (chek == 0)
         {
            printf("Error: Стек переповнено!\n");
            init(&stack);
            return 0;
         }
         if (stack.lp > max)
         {
            max = stack.lp;
         }
      }
```

```
        else if (s[i] == ')')
        {
            chek = pop(&stack);
            if (chek == 0)
            {
                printf("Error: Неправильне вкладення!\n");
                init(&stack);
                return 0;
            }
        }
    }

    if (stack.lp == 0)
    {
        printf("Дужки вкладені правильно!");
        printf("\nМаксимальна глибина вкладення: %i\n", max);
    }
    else
    {
        printf("Дужки вкладені НЕ правильно!");
    }
    return 0;
}
```

Результати тестування програми 1:

1.

```
Рядок для перевірки: ((()()))

Дужки вкладені правильно!
Максимальна глибина вкладення: 3
```

2.

```
Рядок для перевірки: a*(b-c:((d+e)*(f+g)))-h*(k-l))+m

Error: Неправильне вкладення!
```

3.

```
Рядок для перевірки: a*(b-c:((d+e)*(f+g)))-h*(k-l)+m

Error: Стек переповнено!
```

Текст програми 2:

```
#include <stdio.h>
#include <windows.h>

typedef struct Node
```

```
{
   char data;
   struct Node *next;
} Node;

typedef struct
{
   Node *main;
   int lp;
} Stack;

void init(Stack *s)
{
   s->main = 0;
   s->lp = 0;
}

int is_full(const Stack *s)
{
   (void)s;
   return 0;
}

int is_empty(const Stack *s) { return s->main == 0; }

int push(Stack *s, char x)
{
   Node *n = (Node *)malloc(sizeof(Node));
   n->data = x;
   n->next = s->main;
   s->main = n;
   s->lp++;
   return 1;
}

int pop(Stack *s)
{
   if (is_empty(s))
   {
      printf("Error: Стек пустий!\n");
      return 0;
   }
```

```
    Node *t = s->main;
    s->main = t->next;
    free(t);
    s->lp--;
    return 1;
}

int peek(const Stack *s, char *val)
{
    if (is_empty(s))
    {
        return 0;
    }
    *val = s->main->data;
    return 1;
}

int main()
{
    SetConsoleOutputCP(65001);

    Stack stack;

    // char s[]={'(', '(', '(', ')', '(', ')', ')',')','\0'};
    char s[] = "a*(b-c:((d+e)*(f+g)))-h*(k-l)+m";
    // char s[]="a*(b-)c:((d+e)*(f+g)))-h*(k(-l)+m";

    int max = 0, chek = 0;
    // char c;
    init(&stack);

    printf("Рядок для перевірки: ");
    puts(s);
    printf("\n");

    for (int i = 0; s[i] != '\0'; i++)
    {
        // peek(&stack, &c);
        // printf("val: %c\n", c);
        if (s[i] == '(')
        {
            chek = push(&stack, s[i]);
            if (chek == 0)
```

```
            {
                printf("Error: Стек переповнено!\n");
                return 0;
            }
            if (stack.lp > max)
            {
                max = stack.lp;
            }
        }
        else if (s[i] == ')')
        {
            chek = pop(&stack);
            if (chek == 0)
            {
                printf("Неправильне вкладення!\n");
                return 0;
            }
        }
    }

    if (stack.lp == 0)
    {
        printf("Дужки вкладені правильно!");
        printf("\nМаксимальна глибина вкладення: %i\n", max);
    }
    else
    {
        printf("Дужки вкладені НЕ правильно!");
    }
    return 0;
}
```

Результати тестування програми 2:

1.

```
Рядок для перевірки: ((()()))

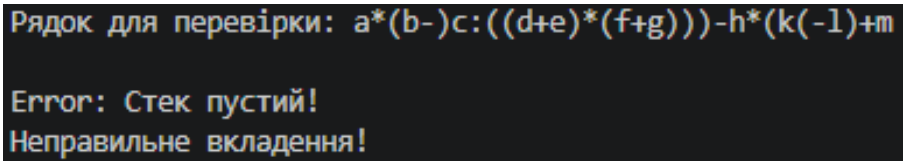
Дужки вкладені правильно!
Максимальна глибина вкладення: 3
```

2.

```
Рядок для перевірки: a*(b-c:((d+e)*(f+g)))-h*(k-l)+m

Дужки вкладені правильно!
Максимальна глибина вкладення: 3
```

3.



Висновок:

Отже, під час виконання лабораторної роботи «Стеки та черги» я засвоїв теоретичний матеріал та набув практичних навичок реалізації й застосування абстрактних структур даних типу стек двома способами: одновимірний масив фіксованого розміру та однозв’язний список. У масивній реалізації розмір обмежується розміром масиву та не потребує виділення пам’яті для кожного елемента окремо. В однозв’язному списку розмір не обмежений, крім пам’яттю комп’ютера. Але потребує виділення пам’яті для кожного елемента. А операції у обох реалізаціях виконуються за сталий час. Тому масивну реалізацію слід використовувати за умови відомій кількості елементів, а однозв’язний список коли елементи мають виділятися динамічно.